SOFTWARE REQUIREMENTS SPECIFICATION

for

Mini Operating System for Machine Learning Inference

Version 1.0

Prepared by:
Piyush Singh Bhati (B24CS1101)
Baviskar Darpan Chandrashekhar (B24CS1020)
Harshit (B24CS1031)
Aashma (B24CS1002)

Submitted to: Dr. Romi Banerjee

January 21, 2026

Contents

1 Introduction **3**
- 1.1 Purpose . . . 3
- 1.2 Who's Going to Read This . . . 3
- 1.3 What We're Building (and What We're Not) . . . 3

2 Overall Description **5**
- 2.1 Why We're Doing This . . . 5
- 2.2 Who Would Use This . . . 5
- 2.3 What the System Actually Does . . . 6
- 2.4 Hardware Requirements . . . 7
- 2.5 Our Design Philosophy . . . 7
- 2.6 How People Would Use It . . . 8

3 System Features **9**
- 3.1 What's Important (Priorities) . . . 9
  - 3.1.1 Stuff We Absolutely Need (High Priority) . . . 9
  - 3.1.2 Stuff We Really Should Have (Medium Priority) . . . 9
  - 3.1.3 Nice-to-Have Stuff (Low Priority) . . . 9
- 3.2 Specific Things the System Has to Do . . . 10
  - 3.2.1 Hardware Requirements . . . 10
  - 3.2.2 Graph Processing Requirements . . . 10
  - 3.2.3 Execution Requirements . . . 10
  - 3.2.4 Memory Requirements . . . 11
  - 3.2.5 Communication Requirements . . . 11

4 Other Requirements (Non-Functional Stuff) **13**
- 4.1 Performance Requirements . . . 13
- 4.2 Predictability Requirements . . . 13
- 4.3 Security Requirements . . . 13
- 4.4 Reliability Requirements . . . 13
- 4.5 Power Requirements . . . 13
- 4.6 Quality Stuff We Want . . . 14
- 4.7 Business/Project Rules . . . 14

5 Other Requirements **15**
- 5.1 Documentation We Need . . . 15
- 5.2 Development Constraints . . . 15
- 5.3 Compliance Requirements . . . 15
- 5.4 Appendices . . . 15
  - 5.4.1 Appendix A: Glossary (Terms We Use) . . . 15
  - 5.4.2 Appendix B: Acronyms . . . 16

1 Introduction

Purpose

This document describes what we need to build for our Mini OS project that's meant specifically for running ML inference on ARM chips. Basically, we're making a super lightweight OS that only does one thing: runs machine learning models really fast and predictably. Unlike regular OSes like Linux that have to do everything for everyone, our OS is specialized just for ML inference. This SRS is our roadmap for what to build and how to test it.

Who's Going to Read This

This document is for:

- **Us (the developers)**: To know exactly what code we need to write
- **Our professors**: To check if we're building the right thing
- **Other students who might want to extend it**: To understand how everything works
- **People testing our system**: To figure out how to test everything properly

What We're Building (and What We're Not)

Our system is a minimal OS for ARM boards that does ML inference. Here's what's included:

Stuff We're Definitely Doing:

- Getting ARM hardware running (bootup, memory setup, interrupts, etc.)
- Taking in ML computation graphs (like neural network models)
- Optimizing those graphs (scheduling, memory planning, combining operations)
- Setting up memory once and for all before running
- Running everything in a predictable, deterministic way
- Basic communication to get graphs in and results out
- Using ARM-specific features like NEON for speed

Stuff We're Definitely NOT Doing:

- Making it POSIX-compatible or like Linux
- Supporting multiple users or processes
- Letting people run random programs
- Adding filesystems or shells
- Doing ML training (only inference!)
- Handling models that change size during runtime
- Full networking stacks

Basically, it's more like firmware for AI chips than a real general-purpose OS. It's for devices that do one thing really well.

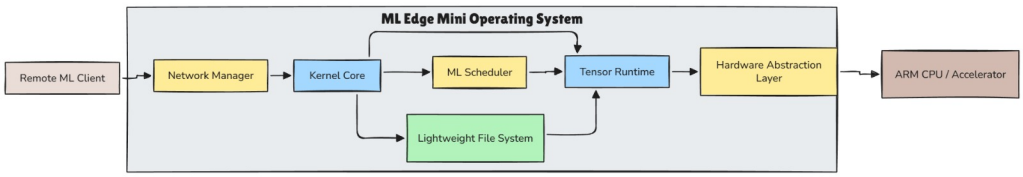

Figure 1.1: How everything fits together - Component Diagram

2 Overall Description

Why We're Doing This

Regular OSes like Linux are made for running lots of different programs, but ML inference is different. It's all about running a fixed computation graph (like a neural network) as fast and predictably as possible. Linux has extra stuff we don't need that slows things down. Our idea is to make an OS that's built around ML graphs from the ground up, so we can control everything and make it super efficient.

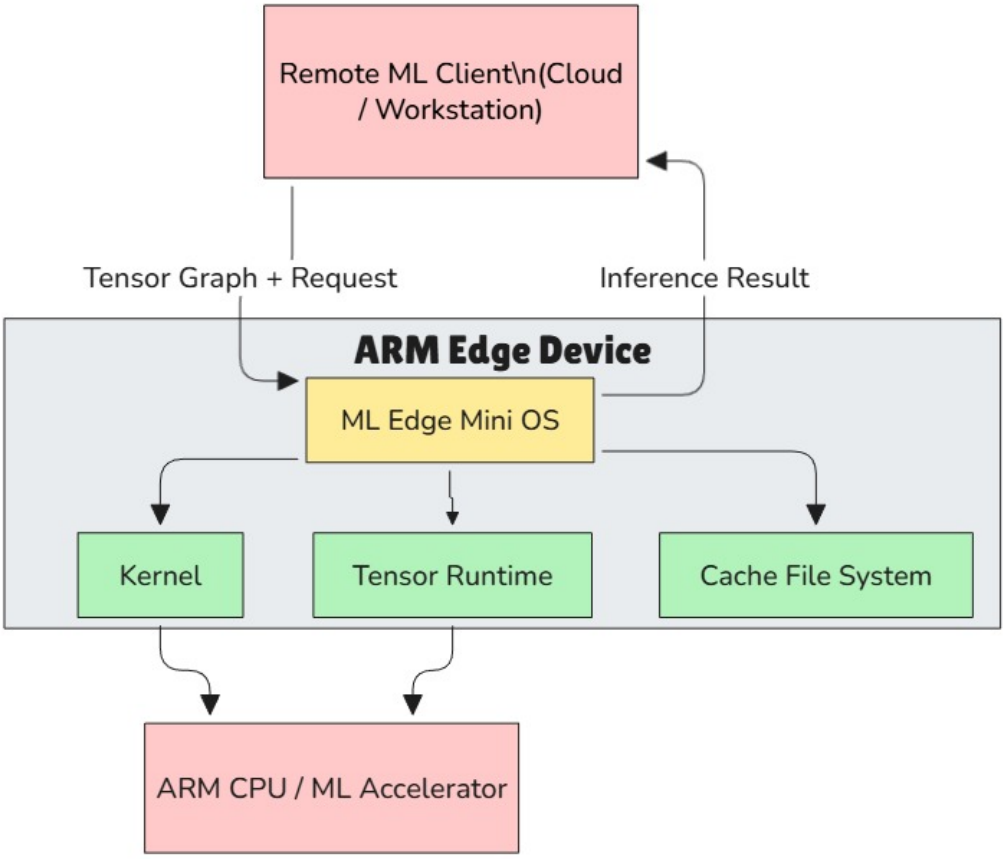

Figure 2.1: Where our system would be used - Deployment Diagram

Who Would Use This

- **Embedded system makers**: People building smart cameras, robots, or other AI devices
- **ML model developers**: People who train models and want to run them on edge devices
- **People deploying AI systems**: Setting up our OS on actual hardware
- **Device manufacturers**: Companies making chips that need inference firmware

What the System Actually Does

Here are the main jobs our OS has to do:

1. Hardware Setup Stuff

- Boot up ARM chips and initialize everything
- Set up memory management (MMU)
- Handle caches properly
- Deal with interrupts and timers
- Work with different ARM chips (Cortex-A, Cortex-M, etc.)

2. Graph Processing Pipeline

- Take in ML computation graphs (our models)
- Check if they're valid and safe to run
- Optimize them (schedule operations, plan memory, combine ops)
- Figure out what depends on what
- Make a fixed execution plan

3. Running Stuff Predictably

- Run operations in the right order without switching between them
- Make memory access patterns predictable
- Use ARM NEON instructions for speed
- Guarantee worst-case timing

4. Memory Management

- Plan all memory usage before running (no malloc during inference!)
- Reuse memory when tensors aren't needed anymore
- Arrange memory for good cache performance
- Protect important memory areas

5. Talking to the Outside World

- Simple way to send graphs to the system
- Way to get inference results back
- Report if everything's working okay
- Accept configuration settings

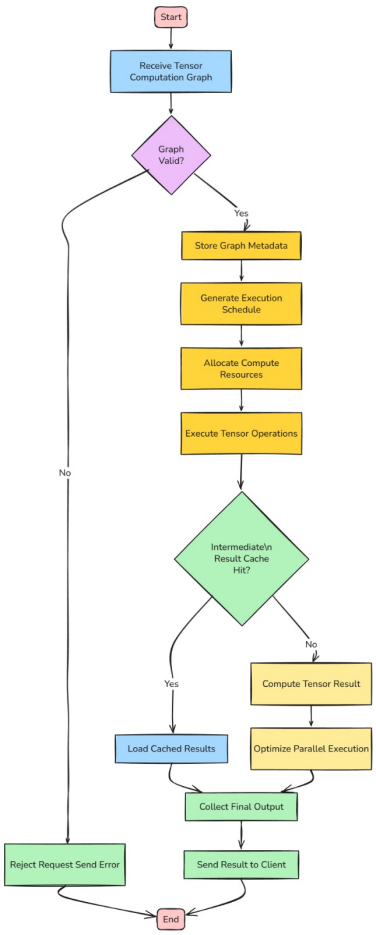

Figure 2.2: How data flows through our system - Activity Diagram

Hardware Requirements

- **Processor**: ARM Cortex-A or Cortex-M chips (the kind in phones and IoT devices)
- **Memory**: At least 16MB RAM, but 64MB+ would be better for real models
- **Storage**: Just enough for the bootloader and config
- **Communication**: Simple stuff like serial, SPI, I2C, or basic Ethernet
- **Power**: Made for battery-powered or low-power devices

Our Design Philosophy

We're building this around a few key ideas:

1. **Graphs are king**: ML computation graphs are what we run, not processes or threads
2. **Plan ahead**: Since we know the whole graph beforehand, we can optimize everything upfront
3. **Predictability over speed**: We care more about consistent timing than being the absolute fastest
4. **Less is more**: We're making it secure and reliable by leaving stuff out, not adding more
5. **We control everything**: Instead of asking Linux nicely to schedule things, we just do it ourselves

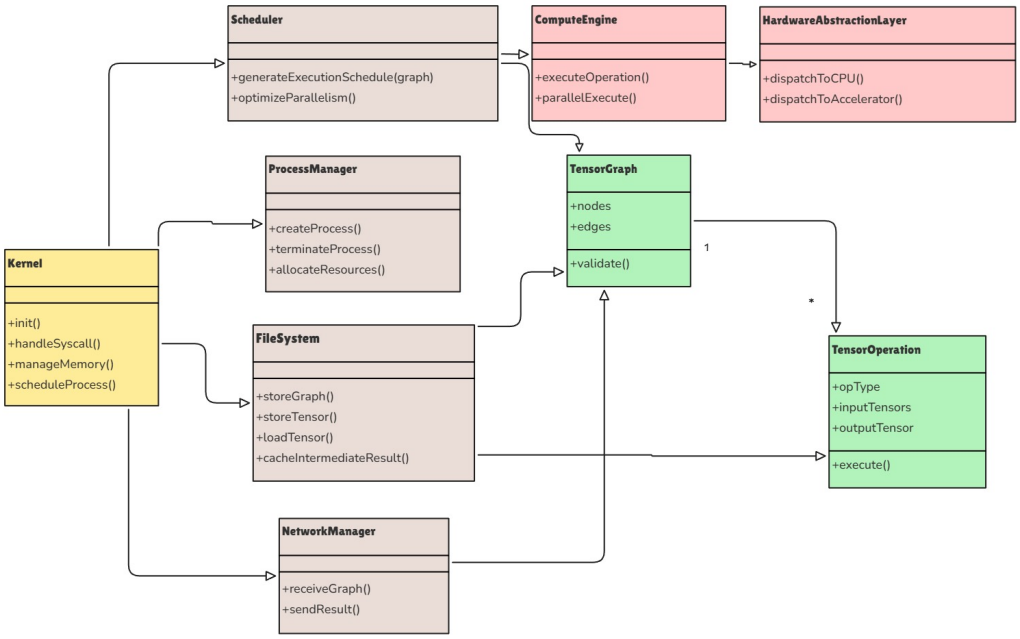

Figure 2.3: Main classes and how they connect - Class Diagram

How People Would Use It

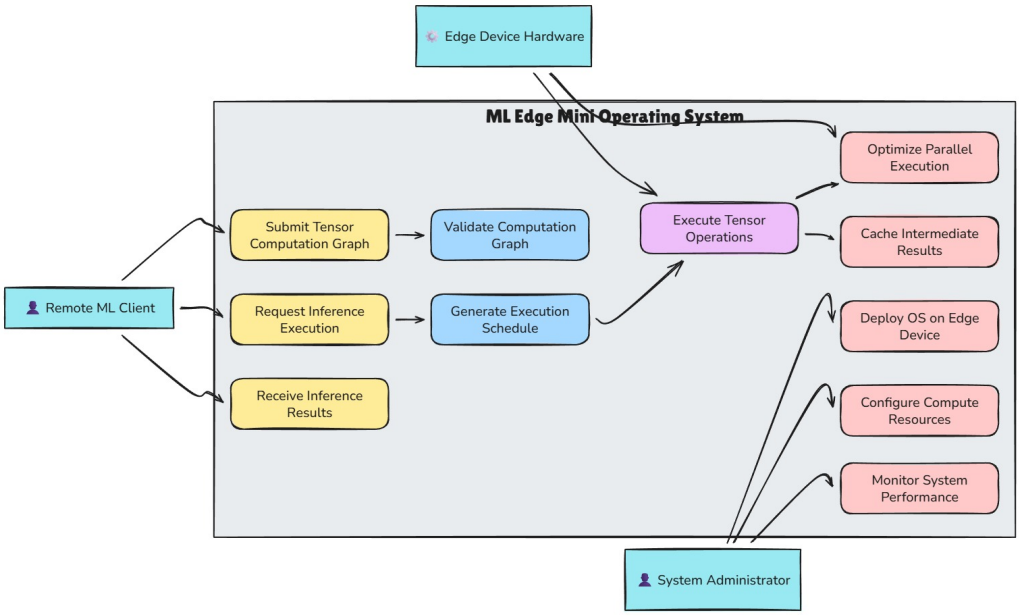

Figure 2.4: Different ways people would interact with our system - Use Case Diagram

3 System Features

What's Important (Priorities)

Stuff We Absolutely Need (High Priority)

1. **Graph Optimizer** (Critical)
   - Schedule operations based on what needs what
   - Plan all memory usage for the whole graph
   - Combine operations when possible
   - ARM-specific optimizations
2. **Predictable Runner** (Critical)
   - Run operations in fixed order (no switching)
   - Guarantee worst-case timing bounds
   - Handle interrupts without messing up timing
   - Watch execution and check everything's okay
3. **ARM Hardware Layer** (Critical)
   - Start up and configure ARM chips
   - Set up memory protection
   - Control caches properly
   - Manage interrupt controller

Stuff We Really Should Have (Medium Priority)

4. **Memory Manager** (High Priority)
   - Allocate all memory once before running
   - Reuse memory when tensors aren't needed
   - Arrange memory for good cache hits
   - Protect system memory
5. **Communication Stuff** (Medium Priority)
   - Simple way to send graphs to the system
   - Way to get results back
   - Report system status
   - Accept configuration

Nice-to-Have Stuff (Low Priority)

6. **Performance Monitoring** (Low Priority)
   - Measure how long things take
   - Track memory usage
   - Read performance counters
   - Estimate power use

7. **Multiple Models** (Low Priority)
   - Run different models one after another
   - Switch between models
   - Share memory between models

Specific Things the System Has to Do

Hardware Requirements

1. **FR-001**: Start up ARM processor cores properly (including CP15 registers)
2. **FR-002**: Set up MMU with just enough page tables
3. **FR-003**: Turn on and manage data/instruction caches
4. **FR-004**: Handle hardware interrupts with predictable timing
5. **FR-005**: Provide timer services for timing execution

Graph Processing Requirements

6. **FR-006**: Accept computation graphs in a simple binary format
7. **FR-007**: Check if graphs are valid and operators are supported
8. **FR-008**: Figure out which operations depend on which
9. **FR-009**: Create a fixed execution schedule
10. **FR-010**: Plan all memory usage for the whole graph

Execution Requirements

11. **FR-011**: Run operators in dependency order (no preemption)
12. **FR-012**: Guarantee worst-case execution time bounds
13. **FR-013**: Use ARM NEON SIMD instructions where it helps
14. **FR-014**: Make memory access patterns predictable
15. **FR-015**: Handle errors without breaking timing guarantees

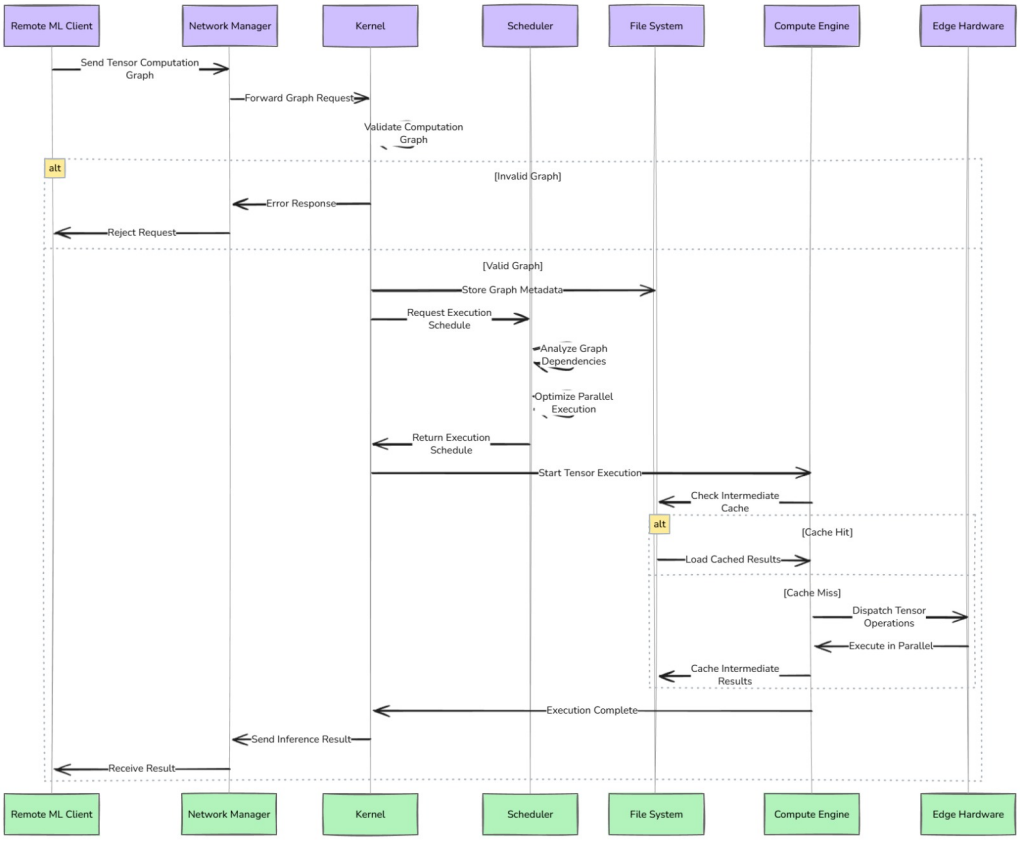

Figure 3.1: Step-by-step execution - Sequence Diagram

Memory Requirements

16. **FR-016**: Allocate all memory before starting execution
17. **FR-017**: Reuse memory for tensors that don't overlap in time
18. **FR-018**: Arrange memory for best cache performance
19. **FR-019**: Protect critical system memory
20. **FR-020**: Provide memory usage info

Communication Requirements

21. **FR-021**: Have a simple interface for graph input
22. **FR-022**: Output inference results
23. **FR-023**: Report system status (working/not working)
24. **FR-024**: Accept configuration parameters
25. **FR-025**: Validate all inputs

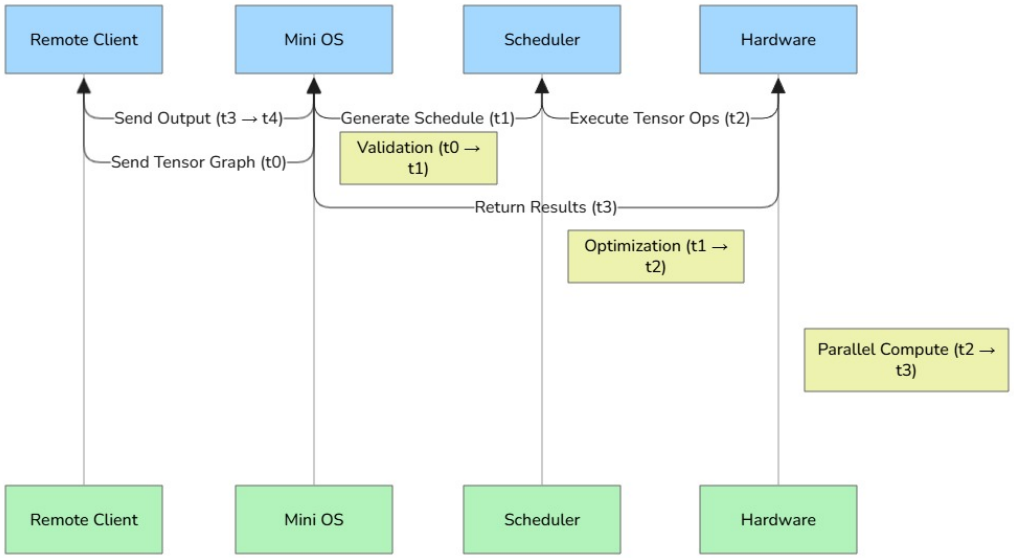

Figure 3.2: Timing guarantees - Timing Diagram

4 Other Requirements (Non-Functional Stuff)

Performance Requirements

1. **PR-001**: Be at most 50% slower than running directly on bare metal
2. **PR-002**: Add less than 5% overhead to total inference time
3. **PR-003**: Optimize graphs in under 100ms for 1000 operations
4. **PR-004**: Allocate memory in under 10ms for typical models
5. **PR-005**: Handle interrupts in under 10 microseconds
6. **PR-006**: If we switch contexts, do it in under 1 microsecond

Predictability Requirements

1. **DR-001**: Execution time shouldn't vary by more than 5% between identical runs
2. **DR-002**: Memory access patterns should be the same every time
3. **DR-003**: Cache behavior should be predictable
4. **DR-004**: Interrupts shouldn't break timing guarantees
5. **DR-005**: Give worst-case time bounds for each operator

Security Requirements

1. **SR-001**: Check input graphs for safety
2. **SR-002**: Protect important memory areas
3. **SR-003**: No loading random code at runtime
4. **SR-004**: If in untrusted environment, authenticate communication
5. **SR-005**: No internet access by default

Reliability Requirements

1. **RR-001**: Run for 1000 hours straight without crashing
2. **RR-002**: Detect memory corruption 99.9% of the time
3. **RR-003**: Recover from non-fatal errors without restarting
4. **RR-004**: Check all critical operations
5. **RR-005**: Report system health

Power Requirements

1. **PWR-001**: Sleep between inferences to save power
2. **PWR-002**: Power use should be predictable
3. **PWR-003**: Be energy efficient per inference
4. **PWR-004**: Turn off clocks for idle parts
5. **PWR-005**: Power saving shouldn't break timing

Quality Stuff We Want

- **Maintainable**: Code should be organized, commented, and modular
- **Testable**: We should be able to test everything properly
- **Portable**: Should work on different ARM chips with minimal changes
- **Verifiable**: Timing guarantees should be provable
- **Simple**: Keep it as simple as possible

Business/Project Rules

1. **BR-001**: Should be usable commercially (in theory)
2. **BR-002**: Work with standard ML model formats
3. **BR-003**: Be possible to certify for safety-critical uses
4. **BR-004**: No proprietary dependencies
5. **BR-005**: Should work with custom AI accelerators

5 Other Requirements

Documentation We Need

1. **DOC-001**: Complete API docs for all interfaces
2. **DOC-002**: Guide for putting it on different ARM boards
3. **DOC-003**: Guide for tuning performance
4. **DOC-004**: Security guide for sensitive uses
5. **DOC-005**: Troubleshooting guide

Development Constraints

1. **DC-001**: Write it in C and ARM assembly
2. **DC-002**: No dynamic memory allocation at runtime
3. **DC-003**: Core system under 64KB
4. **DC-004**: No floating point in kernel code
5. **DC-005**: Timing-critical code should be analyzable for worst-case timing

Compliance Requirements

1. **COMP-001**: Follow ARM Architecture Reference Manual
2. **COMP-002**: Support common ML operators
3. **COMP-003**: Work with common embedded toolchains (GCC, etc.)
4. **COMP-004**: Work with standard communication protocols
5. **COMP-005**: Be certifiable for safety standards

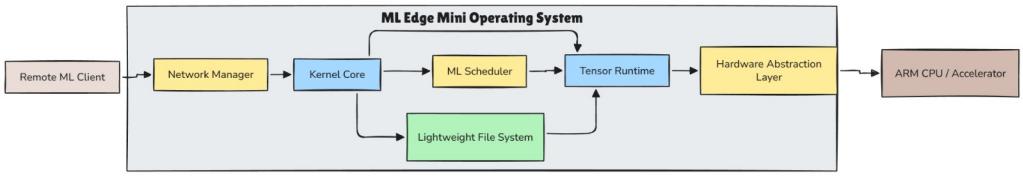

Figure 5.1: Component interactions - Component Diagram (also used for composite structure)

Appendices

Appendix A: Glossary (Terms We Use)

- **Computation Graph**: ML model as a graph of operations (like a neural network)
- **Deterministic Execution**: Running with predictable, repeatable timing
- **Static Scheduling**: Figuring out execution order beforehand
- **Whole-Graph Optimization**: Optimizing the entire model at once
- **Tensor Lifetime**: How long a tensor (data blob) is needed in memory

- **WCET**: Worst-Case Execution Time (longest something could take)

Appendix B: Acronyms

- **SRS**: Software Requirements Specification (this document!)
- **ML**: Machine Learning
- **OS**: Operating System
- **MMU**: Memory Management Unit
- **SIMD**: Single Instruction Multiple Data (doing multiple things at once)
- **NEON**: ARM's SIMD instructions
- **WCET**: Worst-Case Execution Time
- **HAL**: Hardware Abstraction Layer